

Project Documentation

Resumo do Projeto: Implementação do Padrão Factory Method

Este projeto demonstra a aplicação do padrão de design Factory Method para a criação de diferentes tipos de mobilia (`IMobilia`). O objetivo é fornecer uma estrutura flexível e extensível para a criação de objetos de mobilia sem acoplar o código cliente às classes concretas.

Estrutura do Projeto

1. Interfaces e Entidades

- `IMobilia` : Interface que define as propriedades e métodos comuns a todas as mobílias.

```
public interface IMobilia
{
    string Name { get; }
    decimal Price { get; }
    string Material { get; }
    string Color { get; }
}
```

- `IMobiliaFactory` : Interface que define o método `CreateMobilia` para criar objetos `IMobilia` .

```
public interface IMobiliaFactory
{
    IMobilia CreateMobilia();
}
```

2. Fábricas Concretas

- `SofaFactory` : Implementa `IMobiliaFactory` para criar objetos `Sofa` .

```
public class SofaFactory : IMobiliaFactory
{

```

```
public IMobilia CreateMobilia() ⇒ new Sofa();
}
```

- **TableFactory** : Implementa **IMobiliaFactory** para criar objetos **Table** .

```
public class TableFactory : IMobiliaFactory
{
    public IMobilia CreateMobilia() ⇒ new Table();
}
```

- **ChairFactory** : Implementa **IMobiliaFactory** para criar objetos **Chair** .

```
public class ChairFactory : IMobiliaFactory
{
    public IMobilia CreateMobilia() ⇒ new Chair();
}
```

3. Factory Creator

- **MobiliaFactoryCreator** : Classe que centraliza a criação das fábricas de mobilia. Contém um dicionário que mapeia strings para instâncias de **IMobiliaFactory** e fornece o método **GetMobiliaFactoryByString** para obter a fábrica correta com base em uma string.

```
public class MobiliaFactoryCreator : IMobiliaFactoryCreator
{
    private readonly Dictionary<string, IMobiliaFactory> _mobiliaFactories;

    public MobiliaFactoryCreator(Dictionary<string, IMobiliaFactory> mobiliaFactories)
    {
        _mobiliaFactories = mobiliaFactories;
    }

    public IMobiliaFactory GetMobiliaFactoryByString(string mobilia)
        ⇒ _mobiliaFactories.ContainsKey(mobilia) ? _mobiliaFactories[mobilia] : null;
}
```

```

        mobilia] : throw new ArgumentException("Mobilia not found");
    }

```

4. Modelo Principal

- **MainModel** : Classe que utiliza **MobiliaFactoryCreator** para converter uma lista de strings em uma lista de objetos **IMobilia** e delega a criação de relatórios para **ICreateMobiliaRelatory** .

```

public class MainModel : IMainModel
{
    private readonly IMobiliaFactoryCreator _mobiliaFactoryCreator;
    private readonly IMobiliaAdapter _mobiliaAdapter;
    private readonly ICreateMobiliaRelatory _createMobiliaRelatory;

    public MainModel(IMobiliaFactoryCreator mobiliaFactory, IMobiliaAd
apter mobiliaAdapter, ICreateMobiliaRelatory createMobiliaRelatory)
    {
        _mobiliaFactoryCreator = mobiliaFactory;
        _mobiliaAdapter = mobiliaAdapter;
        _createMobiliaRelatory = createMobiliaRelatory;
    }

    public Task<List<IMobilia>> ConvertMobiliaListStringToIMobiliaList(Li
st<string> mobList)
        => Task.Run(() => _mobiliaAdapter.ConvertMobiliaListStringToIMobil
iaList(mobList, _mobiliaFactoryCreator));

    public Task CreateRelatory(List<IMobilia> mobList)
        => _createMobiliaRelatory.CreateRelatoryByList(mobList);
}

```

5. Serviços

- **CreateMobiliaRelatory** : Classe que cria um relatório a partir de uma lista de objetos **IMobilia** , utilizando um **IRelatoryFormatter** .

```

public class CreateMobiliaRelatory : ICreateMobiliaRelatory
{
    private readonly IRelatoryFormatter _formatter;

    public CreateMobiliaRelatory(IRelatoryFormatter formatter)
    {
        _formatter = formatter;
    }

    public Task CreateRelatoryByList(List<IMobilia> mobList)
    {
        using StreamWriter streamWriter = new StreamWriter("Mobilia.txt");
        _formatter.Format(streamWriter, mobList);
        return Task.CompletedTask;
    }
}

```

6. Adaptadores

- **MobiliaAdapter** : Classe que converte uma lista de strings em uma lista de objetos **IMobilia** .

```

public class MobiliaAdapter : IMobiliaAdapter
{
    public List<IMobilia> ConvertMobiliaListStringToIMobiliaList(List<string> mobiliaStringList, IMobiliaFactoryCreator mobiliaFactoryCreator)
    {
        var convertedMobList = new List<IMobilia>();

        foreach (var mobilia in mobiliaStringList)
        {
            if (!string.IsNullOrEmpty(mobilia))
                convertedMobList.Add(mobiliaFactoryCreator.GetMobiliaFactoryByString(mobilia).CreateMobilia());
        }
        return convertedMobList;
    }
}

```

```
}  
}
```

7. Apresentador

- **MainPresenter** : Classe que coordena a interação entre a visão (**IMainView**) e o modelo (**IMainModel**).

```
public class MainPresenter  
{  
    private readonly IMainView _view;  
    private readonly IMainModel _model;  
  
    public MainPresenter(IMainView view, IMainModel model)  
    {  
        _view = view;  
        _view.SendMobilia += SendMobAsync;  
  
        _model = model;  
    }  
  
    private async void SendMobAsync()  
    {  
        try  
        {  
            var IMobiliaList = await _model.ConvertMobiliaListStringToIMobil  
iaList(_view.MobListInString);  
            await _model.CreateRelatory(IMobiliaList);  
            _view.ShowMessageBox("Mobilia list sent successfully", "Succe  
ss", MessageBoxIcon.Information, MessageBoxButtons.OK);  
        }  
        catch (Exception ex)  
        {  
            _view.ShowMessageBox(ex.Message, "Error", MessageBoxIcon.  
Error, MessageBoxButtons.OK);  
        }  
    }  
}
```

8. Visão

- **MainView** : Classe que gerencia a interface do usuário e interage com o usuário.

```
public partial class MainView : Form, IMainView
{
    public MainView() ⇒ InitializeComponent();

    public List<string> MobListInString { get ⇒ IstMob.Items.Cast<string>().ToList(); }

    public event Action? SendMobilia;

    public void ShowMessageBox(string message, string caption, MessageBoxIcon icon, MessageBoxButtons buttons)
        ⇒ MessageBox.Show(message, caption, buttons, icon);

    private void btnAdd_Click(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(cmbMob.SelectedText))
        {
            ShowMessageBox("Please select a mobilia", "Error", MessageBoxIcon.Error, MessageBoxButtons.OK);
            return;
        }
        IstMob.Items.Add(cmbMob.SelectedText);
    }

    private void btnRemove_Click(object sender, EventArgs e) ⇒ IstMob.Items.Remove(IstMob.SelectedItem ?? string.Empty);
    private void btnSend_Click(object sender, EventArgs e) ⇒ SendMobilia?.Invoke();
    private void Exit(object sender, EventArgs e) ⇒ Application.Exit();
}
```

Análise dos Princípios SOLID

Princípio da Responsabilidade Única (SRP)

- **MainModel** : Coordena a conversão de listas de strings em listas de objetos **IMobilia** e a criação de relatórios. Está de acordo com o SRP, pois sua responsabilidade é a coordenação.
- **CreateMobiliaRelatory** : Cria um relatório a partir de uma lista de objetos **IMobilia** . Está de acordo com o SRP.
- **MobiliaAdapter** : Converte uma lista de strings em uma lista de objetos **IMobilia** . Está de acordo com o SRP.
- **MainPresenter** : Coordena a interação entre a visão e o modelo. Está de acordo com o SRP.
- **MainView** : Gerencia a interface do usuário e interage com o usuário. Está de acordo com o SRP.

Princípio Aberto/Fechado (OCP)

- **MainModel** : Está aberta para extensão através das interfaces, sem necessidade de modificar a classe existente. Está de acordo com o OCP.
- **CreateMobiliaRelatory** : Está aberta para extensão através da interface **IRelatoryFormatter** . Está de acordo com o OCP.
- **MobiliaAdapter** : Está aberta para extensão através da interface **IMobiliaFactoryCreator** . Está de acordo com o OCP.
- **MainPresenter** : Está aberta para extensão através das interfaces **IMainView** e **IMainModel** . Está de acordo com o OCP.
- **MainView** : Está aberta para extensão através da interface **IMainView** . Está de acordo com o OCP.

Princípio de Substituição de Liskov (LSP)

- **MainModel** : Implementa a interface **IMainModel** , permitindo substituição por qualquer implementação dessa interface. Está de acordo com o LSP.
- **CreateMobiliaRelatory** : Implementa a interface **ICreateMobiliaRelatory** , permitindo substituição por qualquer implementação dessa interface. Está de acordo com o LSP.
- **MobiliaAdapter** : Implementa a interface **IMobiliaAdapter** , permitindo substituição por qualquer implementação dessa interface. Está de acordo com o LSP.

- **MainPresenter** : Utiliza as interfaces **IMainView** e **IMainModel** , permitindo substituição por qualquer implementação dessas interfaces. Está de acordo com o LSP.
- **MainView** : Implementa a interface **IMainView** , permitindo substituição por qualquer implementação dessa interface. Está de acordo com o LSP.

Princípio da Segregação de Interface (ISP)

- **IMainModel** : Define apenas os métodos necessários para a conversão e criação de relatórios. Está de acordo com o ISP.
- **ICreateMobiliaRelatory** : Define apenas os métodos necessários para a criação de relatórios. Está de acordo com o ISP.
- **IMobiliaAdapter** : Define apenas os métodos necessários para a conversão de listas de strings em listas de objetos **IMobilia** . Está de acordo com o ISP.
- **IMainView** : Define apenas os métodos necessários para a interação com a interface do usuário. Está de acordo com o ISP.

Princípio da Inversão de Dependência (DIP)

- **MainModel** : Depende das abstrações **IMobiliaFactoryCreator** , **IMobiliaAdapter** e **ICreateMobiliaRelatory** , não de implementações concretas. Está de acordo com o DIP.
- **CreateMobiliaRelatory** : Depende da abstração **IRelatoryFormatter** , não de implementações concretas. Está de acordo com o DIP.
- **MobiliaAdapter** : Depende da abstração **IMobiliaFactoryCreator** , não de implementações concretas. Está de acordo com o DIP.
- **MainPresenter** : Depende das abstrações **IMainView** e **IMainModel** , não de implementações concretas. Está de acordo com o DIP.
- **MainView** : Depende da abstração **IMainView** , não de implementações concretas. Está de acordo com o DIP.

Conclusão

O projeto está bem alinhado com os princípios SOLID. As classes e interfaces têm responsabilidades claras e bem definidas, estão abertas para extensão, permitem substituição por implementações intercambiáveis, definem apenas os métodos necessários e dependem de abstrações em vez de implementações

concretas. No geral, o projeto segue bem os princípios SOLID e está bem estruturado para facilitar a manutenção, a extensibilidade e a testabilidade do código.